



SAPIENZA
UNIVERSITÀ DI ROMA

Interactive Graphics Report
“DinosEscape”

Mario Morra

2156770

1	Introduction	4
1.1	Overview.....	4
1.2	Game Concept	4
1.3	Gameplay Summary	4
2	Development Environment and Libraries	5
2.1	WebGL.....	5
2.2	Three.js.....	5
2.3	GLTFLoader.....	6
2.4	External Assets.....	6
3	Software Architecture	6
3.1	Project Structure	6
3.2	Application Flow.....	7
3.3	Game State Management.....	7
3.4	Asset Loading and Management	8
3.5	Main Game Loop	8
4	Scene Composition	9
4.1	Scene Initialization	9
4.2	Camera System	9
4.3	Rendering Configuration	9
4.4	Atmospheric Elements.....	10
4.5	Ground System.....	10
4.6	Environment Layout.....	10
5	Lighting and Rendering.....	11
5.1	Lighting System	11
5.2	Shadow Mapping.....	12
5.3	Materials and Textures	12
6	Character System.....	13
6.1	T-Rex Model	13

6.2	Skeletal Animation.....	14
6.3	Character Movement	14
7	Gameplay Systems.....	15
7.1	Obstacles and Collectibles	15
7.2	Collision Detection	15
7.3	Score and Lives	15
7.4	Difficulty Progression	15
8	User Interface	16
8.1	Introductory Sequence	16
8.2	Main Menu	16
8.3	Heads-Up Display	17
8.4	Pause and Game Over Screens.....	17
9	Conclusions.....	18

1 Introduction

1.1 Overview

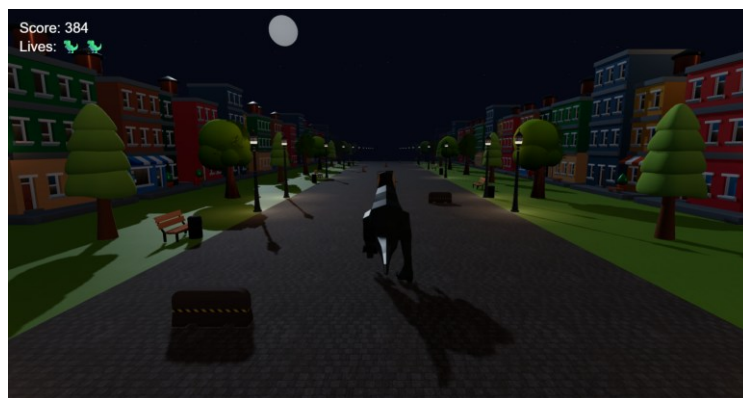
DinosEscape is a browser-based 3D endless runner developed using JavaScript and the Three.js graphics framework. The application places the player in control of a Tyrannosaurus Rex running through a deserted urban environment at night, where survival depends on avoiding obstacles, collecting bonuses, and adapting to an increasingly challenging game pace.

The project combines real-time rendering techniques with interactive gameplay systems to create a complete gaming experience entirely executed within a web browser. Through the integration of 3D models, dynamic lighting, textured environments, skeletal animation, sound effects, and user interface elements, the application demonstrates the capabilities of modern WebGL-based graphics technologies.

1.2 Game Concept

The game is set in a fictional futuristic scenario in which a genetically engineered Tyrannosaurus Rex escapes from a research facility and finds itself running through an abandoned city. The player must guide the creature through the urban environment while avoiding hazards and maintaining its survival for as long as possible.

The visual style focuses on a nighttime atmosphere characterized by moonlight, illuminated street lamps, volumetric depth effects, and dense urban scenery. This setting provides both the narrative context and the visual identity of the application.



1.3 Gameplay Summary

DinosEscape follows the endless runner genre, where the player continuously advances through an apparently infinite environment. The character automatically progresses forward while the player controls lateral movements and jumping actions.

The game world contains different types of obstacles that must be avoided or jumped over. In addition, collectible items are distributed throughout the environment and can be used to recover health and increase the player's score. As the score increases, the difficulty gradually rises through faster movement speeds and more frequent obstacle generation, creating a progressively more demanding gameplay experience.

The application also includes a complete user interface composed of an introductory sequence, a main menu, gameplay heads-up display, pause functionality, and game-over screens, providing a polished and self-contained user experience.

2 Development Environment and Libraries

2.1 WebGL

DinosEscape is a browser-based application built on top of WebGL, the standard graphics API that enables hardware-accelerated 3D rendering within modern web browsers. WebGL provides direct access to the GPU through JavaScript, allowing complex scenes to be rendered in real time without requiring external software or plugins.

Although WebGL offers a powerful low-level graphics interface, developing a complete application directly on top of it would require extensive management of shaders, buffers, transformations, lighting calculations, and rendering pipelines. For this reason, a higher-level framework was adopted to simplify development while still leveraging the performance advantages of GPU acceleration.

2.2 Three.js

The core of the application is built using Three.js, an open-source JavaScript library that provides an abstraction layer over WebGL. Three.js significantly reduces the complexity of 3D graphics programming by offering a rich set of classes and utilities for scene management, rendering, cameras, lights, materials, geometries, textures, and animations.

Within DinosEscape, Three.js is responsible for the creation and management of the entire rendering pipeline. The library is used to construct the game scene, configure the camera system, manage lighting sources, load external assets, render shadows, and update animated objects during gameplay.

Several advanced rendering features provided by Three.js are employed throughout the project, including physically based materials, shadow mapping, fog effects, texture repetition, color space management, and tone mapping. These features contribute significantly to the visual quality of the final application.

2.3 GLTFLoader

To import external three-dimensional assets, the project uses the GLTFLoader module provided by Three.js. This loader supports the glTF and GLB formats, which have become widely adopted standards for the distribution of real-time 3D content.

The GLB format was selected because it stores geometry, materials, textures, and hierarchical information within a single binary file, simplifying asset management and deployment. Through GLTFLoader, all environmental objects and the main character model are loaded dynamically during application startup and integrated into the scene graph.

The loader is also used together with a model caching mechanism that prevents the same asset from being loaded multiple times, improving loading performance and reducing memory overhead during gameplay.

2.4 External Assets

The visual appearance of the application relies on several external assets integrated into the rendering pipeline.

The project includes a collection of 3D models representing the urban environment and gameplay elements, including buildings, trees, street lamps, benches, trash cans, obstacles, collectibles, and the main Tyrannosaurus Rex character.

To improve realism, the environment uses multiple texture maps, including diffuse textures, normal maps, and roughness maps. These textures are applied through physically based materials to reproduce surface details such as pavement stones and grass without increasing geometric complexity.

The audio system also relies on external sound assets, including background music, collision sounds, collectible effects, and interface feedback sounds. These elements contribute to the overall atmosphere and improve player immersion during gameplay.

3 Software Architecture

3.1 Project Structure

The application is organized into a modular architecture composed of independent JavaScript modules, each responsible for a specific aspect of the game. This separation improves code readability, maintainability, and scalability while reducing dependencies between gameplay systems and user interface components.

The project is primarily structured around three source files:

- **main.js**, which contains the core game logic, scene initialization, rendering pipeline, gameplay mechanics, character animation, collision detection, and the main update loop.
- **menu.js**, which implements the graphical user interface associated with the main menu, tutorial panel, statistics screen, and game startup procedures.
- **intro.js**, which manages the introductory narrative sequence displayed before entering the main menu.

This modular organization allows each subsystem to be developed independently while maintaining a clear separation of responsibilities.

3.2 Application Flow

The execution flow of the application follows a sequential structure that guides the player from the initial startup sequence to active gameplay.

When the application is launched, the rendering engine, scene, camera, lighting system, materials, and environment assets are initialized. Once the graphical components have been prepared, the introductory sequence is displayed.

After the introduction is completed or skipped by the user, control is transferred to the main menu. From this point, the player can start a new game, access the tutorial section, or view gameplay statistics.

Starting a new game initializes the gameplay systems, resets the score and player state, and activates the main game loop. During execution, the loop continuously updates object positions, animations, collision detection, score progression, and rendering operations until the game ends or is paused.

3.3 Game State Management

The application uses a state-based architecture to control the behavior of the different game phases. At any given time, the application operates within one of several predefined states.

The main states implemented are:

- **Intro**, responsible for the introductory narrative sequence.
- **Menu**, responsible for displaying and managing the main menu interface.
- **Playing**, representing the active gameplay state.
- **Paused**, temporarily suspending gameplay updates while keeping the application active.
- **Game Over**, triggered when the player loses all available lives.

The current state determines which systems are allowed to update during each frame. For example, gameplay logic and object movement are processed only while the game is in the Playing state, while all gameplay updates remain suspended during the Paused state.

This approach simplifies system management and prevents unintended interactions between independent components.

3.4 Asset Loading and Management

The application relies on a large collection of external assets, including three-dimensional models, textures, and audio files. To efficiently manage these resources, a centralized loading system is employed.

Three-dimensional models are imported through the GLTFLoader module and stored using a caching mechanism. Once a model has been loaded for the first time, subsequent requests reuse the cached version instead of performing additional loading operations.

This strategy significantly reduces loading times and minimizes memory consumption, particularly for frequently reused assets such as obstacles, buildings, street lamps, and environmental decorations.

Textures and audio resources are loaded during initialization and remain available throughout the application lifecycle, ensuring smooth runtime performance without visible loading interruptions.

3.5 Main Game Loop

The core of the application is driven by a real-time update loop implemented through the browser's animation frame system.

During each frame, the game loop performs several operations:

- Updating the score and difficulty progression.
- Moving environmental objects and obstacles.
- Updating character animations.
- Processing user input.
- Evaluating collisions.
- Updating active light sources.
- Rendering the final scene.

This continuous cycle creates the illusion of motion and ensures that all gameplay systems remain synchronized with the visual representation displayed on screen.

The update loop runs until the application is closed, while individual gameplay systems are activated or suspended according to the current game state.

4 Scene Composition

4.1 Scene Initialization

The application begins by creating a Three.js scene that acts as the root container for every object rendered during gameplay. All environmental elements, lights, models, effects, and interactive objects are attached to this scene graph and managed through a hierarchical structure.

To reinforce the nocturnal atmosphere of the game, the scene background is initialized using a dark blue color palette. This color choice provides visual consistency with the moonlit urban environment and contributes to the overall mood of the experience.

Additionally, a fog effect is applied to the entire scene. Fog gradually reduces the visibility of distant objects, improving depth perception and helping to conceal the boundaries of the procedurally recycled environment.

4.2 Camera System

The game uses a perspective camera positioned behind and slightly above the player character. This configuration provides a third-person viewpoint that allows the player to observe both the character and the upcoming obstacles.

Unlike a static camera, the camera dynamically follows the T-Rex during gameplay. Horizontal movements are smoothly interpolated to avoid abrupt transitions when the player changes lane.

The camera continuously adjusts its orientation to keep the character centered within the viewport while maintaining visibility of the road ahead. This approach provides a stable and intuitive perspective that supports fast-paced gameplay.

4.3 Rendering Configuration

Rendering operations are performed through the Three.js WebGL renderer. Several configuration options are enabled to improve visual quality and realism.

Antialiasing is activated to reduce jagged edges and improve the appearance of geometric objects. The renderer also employs sRGB color management to ensure accurate color reproduction across different devices and displays.

To further enhance image quality, the application uses ACES Filmic Tone Mapping. This technique improves the handling of high-intensity light sources and produces a more cinematic appearance by simulating the response of real-world cameras to varying lighting conditions.

Shadow mapping is also enabled, allowing dynamic objects and environmental elements to cast and receive shadows throughout the scene.

4.4 Atmospheric Elements

Several atmospheric components are included to strengthen the visual identity of the game world.

A large moon is positioned in the sky and acts as both a visual element and a reference point for the primary directional light source used throughout the scene. The moon contributes significantly to the nighttime atmosphere and helps establish the environmental context of the game.

In addition, a procedural star field is generated during initialization. Hundreds of stars are randomly distributed throughout the sky using a point-based rendering approach. This technique provides visual richness while maintaining a very low computational cost.

The combination of moonlight, stars, and atmospheric fog creates a coherent environment that supports the game's narrative setting.

4.5 Ground System

The road on which the T-Rex runs is constructed using textured plane geometries positioned horizontally beneath the player. Instead of relying on a single large surface, the ground is divided into multiple segments.

Each segment contains both the main road and adjacent grassy areas located on either side. This structure allows different materials and texture configurations to be applied to distinct regions of the environment.

To create the illusion of an endless world, ground segments are continuously repositioned once they move beyond the visible area. This recycling mechanism eliminates the need to generate new geometry during gameplay and significantly improves performance.

The result is a seamless infinite track that remains visually consistent regardless of the distance traveled by the player.

4.6 Environment Layout

The urban environment is constructed through the repeated placement of decorative and architectural assets along both sides of the road.

The scenery includes:

- Buildings
- Street lamps
- Trees
- Benches
- Trash cans

These elements are distributed throughout the environment to create visual variety and reinforce the impression of a deserted city.

Rather than generating random scenery during gameplay, environmental objects are placed according to predefined spatial patterns. This approach ensures visual consistency while still providing a sufficiently diverse environment.

As with the ground system, environmental assets are continuously recycled and repositioned when they leave the visible area. This technique allows the application to simulate an effectively infinite city while maintaining a limited number of active objects in memory.

5 Lighting and Rendering

5.1 Lighting System

The lighting system combines multiple light sources to create a realistic nighttime environment. A low-intensity ambient light provides global illumination, preventing excessively dark areas, while a hemisphere light simulates indirect light coming from the sky and the surrounding environment.

The primary light source is represented by a directional light that simulates moonlight. This light illuminates the entire scene and is responsible for generating most of the visible shadows, contributing significantly to the atmosphere of the game.

Additional local illumination is provided by the street lamps distributed throughout the environment. Each lamp is associated with a point light source and a visible glowing bulb, creating localized pools of warm light along the road.

To improve performance, only a limited number of nearby street lights remain active at any given time, while distant lights are disabled until they enter the visible area.

5.2 Dynamic Day-Night Presets

The lighting system supports two distinct environmental configurations: a daytime mode and a nighttime mode. Rather than relying on a static scene setup, the application can dynamically switch between these presets during runtime.

Each preset modifies multiple rendering parameters simultaneously, including background color, fog properties, ambient illumination, directional light intensity, tone mapping exposure, and environmental visibility. Additional visual elements such as stars, the moon, and street lamp illumination are enabled or disabled depending on the selected mode.

The nighttime configuration emphasizes atmospheric lighting through moonlight and active street lamps, while the daytime configuration increases overall scene brightness and reduces the contribution of artificial light sources.

This approach allows the same environment to present significantly different visual characteristics without requiring additional geometry or assets, improving both visual variety and user customization.



5.3 Shadow Mapping

Shadow mapping is used to increase depth perception and visual realism. The moonlight directional light generates dynamic shadows that are projected onto the environment and the player character.

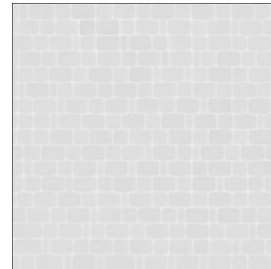
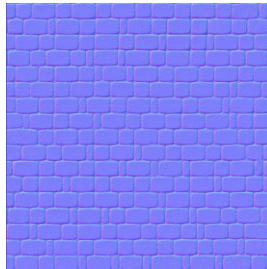
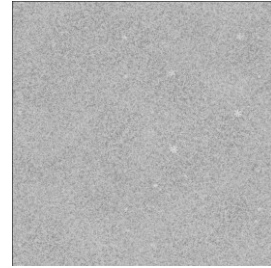
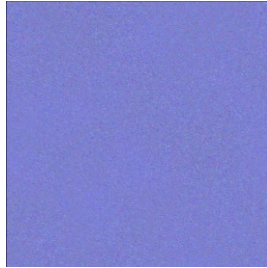
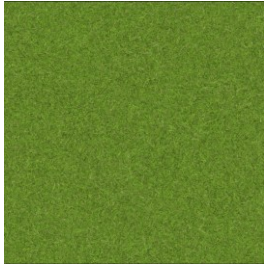
Both static and dynamic objects are configured to cast and receive shadows, allowing environmental elements such as buildings, trees, obstacles, and the T-Rex to interact consistently with the lighting system.

5.4 Materials and Textures

The environment uses physically based materials to improve surface realism. The road and grass areas are textured using diffuse maps, normal maps, and roughness maps.

Diffuse textures define the visible appearance of each surface, while normal maps simulate small geometric details without increasing the complexity of the mesh. Roughness maps control how light is reflected across different materials, producing more convincing visual results.

The combination of physically based materials, dynamic lighting, and shadow mapping contributes to the overall realism and visual quality of the scene.



6 Character System

6.1 T-Rex Model

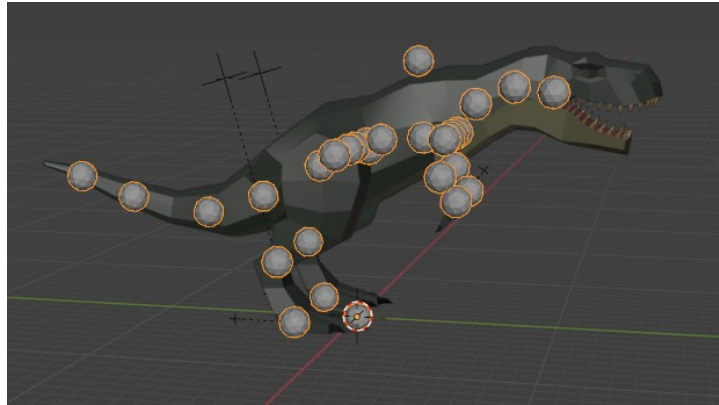
The player character is represented by a three-dimensional Tyrannosaurus Rex model imported in GLB format. Once loaded, the model is integrated into the scene graph and configured to interact with the lighting and shadow systems.



The T-Rex remains at the center of the gameplay experience, while the environment moves relative to its position to create the illusion of continuous forward motion.

6.2 Skeletal Animation

The character animation is based on a skeletal structure composed of multiple interconnected bones. Instead of relying on pre-recorded animations, the movement of the dinosaur is generated procedurally by manipulating specific bones of the skeleton in real time.



The animation system controls the movement of the legs, body, neck, head, and tail, producing a continuous running cycle synchronized with the gameplay speed.

This approach provides greater flexibility compared to static animations and allows the character's movement to dynamically adapt to changes in game speed.

6.3 Character Movement

Player interaction is based on a lane navigation system composed of three possible positions: left, center, and right.

Lane transitions are interpolated smoothly to avoid abrupt movements and improve visual quality. In addition to lateral movement, the character can perform jumps to avoid specific obstacle configurations.

The jump system simulates basic physics through the application of an initial vertical velocity and a gravity factor that gradually returns the character to the ground.

The combination of procedural animation, lane-based navigation, and jump mechanics creates responsive gameplay while maintaining a visually convincing character motion.

7 Gameplay Systems

7.1 Obstacles and Collectibles

The game features multiple obstacle types distributed throughout the environment. Standard obstacles, such as traffic cones and concrete barriers, require the player to change lane in order to avoid collisions.

A special obstacle configuration consisting of a wall of cones can also appear. Since this obstacle occupies all available lanes, the player must perform a jump to pass over it successfully.

In addition to obstacles, collectible bones are periodically spawned along the track. Collecting a bone restores one life, up to the maximum number of lives available, and provides a score bonus.

7.2 Collision Detection

Interactions between the player and game objects are managed through a collision detection system based on distance calculations.

Whenever the T-Rex comes within a predefined distance of an obstacle or collectible item, the corresponding event is triggered. Obstacle collisions reduce the player's remaining lives, while collectible items provide rewards and are removed from the scene.

This approach offers an efficient solution suitable for the relatively simple geometry used in the game.

7.3 Score and Lives

The player begins each game with a limited number of lives. Colliding with obstacles decreases the available lives, while collecting bones can recover lost health.

The score increases continuously as the player survives and progresses through the environment. Additional points are awarded when collectible items are obtained.

The current score and remaining lives are displayed through the in-game heads-up display, providing constant feedback during gameplay.

7.4 Difficulty Progression

To maintain an increasing level of challenge, the game implements a progressive difficulty system linked to the player's score.

As the score grows, the overall speed of the environment increases and obstacles are generated more frequently. This gradually reduces reaction times and forces the player to make faster decisions.

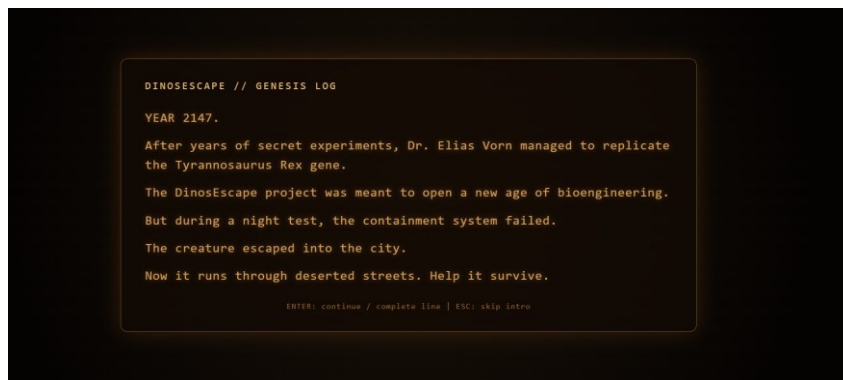
The scaling mechanism ensures that gameplay remains engaging over long sessions while rewarding player skill and concentration.

8 User Interface

The application includes several user interface components designed to guide the player through the different phases of the game while providing constant feedback during gameplay.

8.1 Introductory Sequence

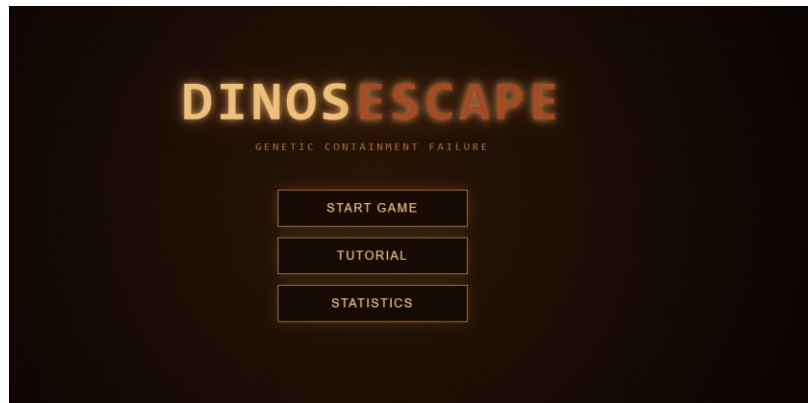
Before accessing the main menu, the player is presented with a narrative introduction that provides the background story of the game. The sequence is displayed using a typewriter-style text effect accompanied by audio feedback, creating a more immersive introduction to the experience. The player can either progress through the story manually or skip the sequence entirely.



8.2 Main Menu

The main menu acts as the central navigation hub of the application. From this screen, players can start a new game, access the tutorial section, or view their best recorded score.

The interface adopts the same visual style used throughout the game, ensuring graphical consistency between menus and gameplay.



8.3 Heads-Up Display

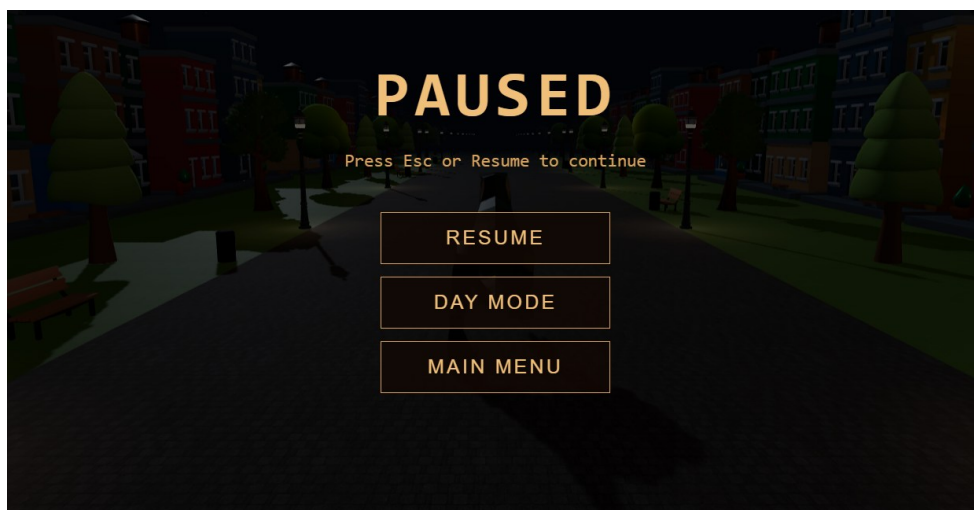
During gameplay, a heads-up display (HUD) provides real-time information about the current game state. The HUD displays the player's score and the number of remaining lives, allowing immediate access to the most relevant gameplay information without interrupting the action.

8.4 Pause and Game Over Screens

The game can be paused at any moment, temporarily suspending gameplay while displaying a dedicated pause menu.

In addition to resuming the current session or returning to the main menu, the pause interface provides a visual mode selector that allows players to switch between daytime and nighttime rendering configurations. The transition is applied immediately and updates the lighting system, atmospheric effects, and environmental appearance without interrupting gameplay.

This functionality provides an additional level of customization while showcasing the flexibility of the rendering system.



9 Conclusions

DinosEscape is a complete browser-based 3D endless runner that combines real-time rendering, dynamic lighting, skeletal animation, collision detection, audio feedback, and interactive gameplay systems within a single application.

The project demonstrates how modern web technologies can be used to create visually rich and responsive interactive experiences without requiring dedicated software installations. Through the integration of Three.js, WebGL, physically based materials, dynamic shadows, and procedural character animation, the application achieves a visually coherent and technically robust result.

From a gameplay perspective, the combination of lane-based movement, jumping mechanics, collectibles, obstacles, and progressive difficulty creates an engaging experience that remains accessible while providing increasing levels of challenge.

Overall, the project successfully integrates graphical techniques and gameplay systems into a cohesive application, delivering both an immersive visual presentation and a complete interactive experience.